UNIVERSITY OF ZAGREB
FACULTY OF ELECTRICAL ENGINEERING AND COMPUTING

MASTER'S THESIS No. 35

Frontier-based Autonomous 3D Exploration for UAVs

Iván David Changoluisa Caiza

Zagreb, June 2023.

Dedicated to my Father, who from heaven continues to guide my path, and to my family waiting for me in Ecuador

Acknowledgement

I would like to express my profound gratitude to my parents, María Carmen and Luis Alfredo, whose invaluable lessons on dedication, hard work, and accountability have been instrumental in shaping my character and aspirations. Their unwavering support has been a foundation for my journey. My heartfelt appreciation also extends to my brothers, Fernando and Victor, for their constant encouragement throughout this process. To my relatives in Spain, your presence and hospitality have made me feel at home in a foreign land, and I am truly thankful. My special thanks to my family and friends in Ecuador who, despite the distance, have always been there, motivating me and making sure I felt loved and remembered.

In academia, my sincere appreciation goes to the IFROS organization, led by Professors Narcis Palomeras and Nuno Gracias from the University of Girona. They provided me with the opportunity to undertake this master's program that unveiled a new world of opportunities, truly making it a life-changing experience.

I am especially grateful to my supervisor, Professor Tamara Petrović from the University of Zagreb. Her insightful guidance was invaluable not only during the development of this thesis but also throughout my tenure over the last two semesters.

My special thanks are reserved for Ana Batinović. Her expert advice and kind assistance were crucial and highly appreciated throughout this project.

My deepest gratitude is extended to Professor Francisco Pérez and Marco Montes from the UPIA lab at CATEC (Centro Avanzado de Tecnologías Aeroespaciales) in Seville, Spain. Their generosity in providing me with the opportunity to conduct my thesis within the company, combined with their tremendous support and facilitation, has been indispensable to the successful completion of my Master. Lastly, I would like to express my gratitude to my IFROS colleagues, whose friendship has made this journey all the more enjoyable. This venture wouldn't have been the same without all of you.

To each one of you who has played a pivotal role in my journey, I offer my sincerest gratitude. Your contributions have truly made a difference. Thank you.

Contents

1. Introduction and Problem definition **1**

2. State of the Art **4**

3. Methodology **7**

3.1. UAV Exploration Framework 7

3.2. Frontier-based Exploration 9

3.2.1. Proposed approach: Collector Exploration 9

3.2.2. Mapping 10

3.2.3. Frontier detection 11

3.2.4. Frontiers analysis 13

3.2.5. Path planning 17

3.2.6. TSP Solver 18

3.3. ROS Architecture 20

4. Results **23**

4.1. TSP Solver Influence 24

4.2. Warehouse Environments 26

4.3. Outdoor Environment 31

4.4. Flight Testing 32

5. Conclusions **35**

6. Appendices **37**

6.1. Appendix A: Detailed ROS Architecture 37

Bibliography **38**

1. Introduction and Problem definition

Autonomous exploration is a crucial task that enables a mobile robot to navigate through unfamiliar environments while simultaneously mapping the surroundings. By acquiring a comprehensive understanding of the environment, the vehicle can navigate safely and efficiently, determining the shortest obstacle-free paths to reach any location within the map. Exploration forms a fundamental component of various mobile robotic applications, including scene reconstruction, search and rescue operations, and continuous monitoring. Initially, autonomous exploration was primarily implemented using Unmanned Ground Vehicles (UGVs), which offered robustness and extended autonomy. However, UGVs were limited to operating in a 2D space, restricting coverage range and the amount of information gathered. With the advancements in Unmanned Aerial Vehicles (UAVs) and their increased autonomous capabilities, along with reduced manufacturing costs, UAVs have emerged as a promising solution to overcome these limitations and significantly expand the boundaries of robotic exploration due to their flexibility and expanded coverage range.

Nevertheless, implementing autonomous exploration with UAVs introduces additional complexity due to the three-dimensional (3D) operating space. This complexity arises from the increased volume of data to process and the wider range of potential locations UAVs can explore. Substantial efforts have been invested in autonomous UAV exploration, resulting in significant advancements for efficiently exploring complex environments. However, many existing methods prioritize exploration time while neglecting map quality assessment. Moreover, most of these approaches adopt a greedy policy that capitalizes on the newly gained knowledge about the environment. In such strategies, exploration targets are selected to maximize specific utilities, such as information gain or travel cost. This approach often leads to local minima, results in redundant information due to back-and-

forth trajectories, and increases the overall travel distance—subsequently escalating the energy consumption of the UAV during exploration.

In this context, this thesis project aims to design and implement an exploration algorithm that enables a UAV to autonomously navigate unexplored environments. This is accomplished by establishing an exploration strategy that not only ensures a fast exploration but also provides a meticulous analysis of the environment, thereby reducing the risk of local minima. The proposed approach uses a LiDAR sensor as the primary source of information. This data informs a frontier-based exploration technique [16] integrated within the OctoMap Framework [9], facilitating efficient and rapid map analysis.

The proposed approach strives to strike a balance between the exploration of the unknown and the exploitation of known map data. The exploitation component, driven by a greedy behavior, seeks to minimize travel distance by targeting the nearest exploration point. Conversely, exploration is facilitated by storing all detected exploration targets throughout the process. As a result, the UAV visits not only clusters with high information gain but also those containing less information, which could potentially uncover important map locations.

The exploration module developed in this project integrates with the UAV's existing localization, navigation, and path-planning modules. This work has been executed using the Robot Operating System (ROS) and tested in a simulated environment using Gazebo. To validate the system's functionality, preliminary flight tests are conducted onboard a quadcopter-type UAV.

This project represents a collaborative endeavor carried out in partnership with CATEC (Centro Avanzado de Tecnologías Aeroespaciales) [1], located in Seville, Spain.

The document is structured as follows: In Section 2, a thorough review of the state of the art in autonomous UAV exploration is conducted. This includes an understanding of the prevailing strategies for autonomous exploration and the positioning of the current work within this context. In section 3, the UAV Exploration Framework is elaborated on, outlining its key components and primary focus. This chapter provides an overview of the algorithms employed for frontiers detection, clusters detection, filtering, and exploration target selection, and details the ROS architecture used in this project. The results obtained from extensive testing in a simulation environment are shared in Section 4. The eval-

[1] http://www.catec.aero/es

uation of three exploration strategies - Greedy, Collector, and Multi-resolution - across four different maps is conducted. The potential benefits of integrating a Travelling Salesman Problem (TSP) Solver are explored initially, followed by the assessment of exploration strategies in warehouse-like environments of varying complexity. The section culminates with the presentation of preliminary results from initial flight tests conducted on a UAV Quadcopter. Finally, Section 5 consolidates the findings of the study, drawing conclusions based on the results obtained and experiences gathered throughout the course of the project.

2. State of the Art

Exploration algorithms aim to identify a set of locations that an autonomous vehicle should visit in order to achieve complete coverage of the desired environment. These algorithms rely on local information gathered by perception sensors such as RGBD cameras or LiDAR scanners [17]. The exploration process occurs incrementally, with the map being constructed as the vehicle navigates the environment. Additionally, an effective exploration framework requires a systematic approach to store the spatial representation of the discovered map. Commonly used mapping techniques include occupancy maps like Euclidean Signed Distance Fields (ESDF) [17], Normal Distribution Transforms [13], and Octomap [9]. Furthermore, exploration algorithms are designed to optimize various utility metrics such as information gain, energy consumption, or time required for exploration. This section provides an overview of the main approaches to autonomous exploration for UAVs.

Over the years, numerous studies have addressed autonomous exploration using UAVs, with most approaches falling into two categories: *Sampling-based* and *Frontier-based* [17].

Sampling-based techniques determine potential viewpoints using sampling methods like RRT or PRM. These techniques offer the following advantages:

- Well-suited for cluttered spaces: Sampling-based approaches are effective in generating viewpoints that can navigate around obstacles and reach unexplored areas in cluttered environments.
- Computational efficiency: These methods are often computationally efficient, enabling real-time planning and exploration, making them suitable for limited payload and light weight onboard computers.

Bircher et al. [2] introduced *Sampling-based* techniques in UAV exploration. Their work employed a receding horizon planning approach, selecting potential viewpoints based on information gain criteria and travel cost. Building upon this

approach, Schmid et al. [14] introduced an informative path planning algorithm inspired by RRT*, expanding the candidate trajectories to achieve full coverage using a single objective function. To accelerate information gain computation, [1] proposed a novel technique based on the Recursive Shadowcasting algorithm and inspired by RRT techniques. The results showed an improved efficiency and speed of the exploration, enabling the vehicle to deal with complex maze type environments.

Although *Sampling-based* approaches are well-suited for cluttered spaces and demand lower computational resources, they can suffer from local minima issues, hindering complete coverage of the target environment [20].

On the other hand, *Frontier-based* approaches offer the following advantages:

- Systematic exploration of unexplored areas: *Frontier-based* methods provide a structured approach to explore the environment by targeting unexplored areas, this facilitate the decision-making algorithm and allows to implement strategies to perform the exploration in a more deterministic way.
- Suitability for large environments: the concept of frontiers allows to identify unexplored spaces in the global map, hence this approach can ensure fully coverage of the space.

Frontier-based exploration, initially introduced by Yamauchi [16] on UGVs applications, has been successfully applied to aerial vehicles. As in the work developed by [21], who successfully applied *Frontier-based* exploration base on the Octomap structure, implementing a scalable frontier detection strategy in large 3D environments. Another example is the work of [3], where the destination frontier is selected to minimize the required velocity gradient for UAV travel. Additionally, [1] addresses the high computational cost associated with frontier exploration using LiDAR scanners by utilizing the multi-resolution capabilities of the Octomap data structure to detect frontiers at a coarse level.

Although *Sampling-based* and *frontier-based* approaches are the prevailing exploration techniques, novel methods have emerged that combine the strengths of each to improve exploration time and computational complexity. For instance, [15] proposed a combination of receding horizon sampling-based and frontier-based methods, enabling the UAV to utilize frontier exploration for global target selection and sampling techniques for local exploration. Furthermore, [18] introduced a *Frontier-based* exploration approach complemented with a Travelling

Salesman Solver from [8] to generate minimum-time trajectories for visiting target locations. This strategy was further extended by [19], adapting the approach for collaborative exploration with a decentralized team of UAVs, thereby reducing the exploration time required for large environments.

In conclusion, exploration algorithms play a vital role in autonomous vehicle systems, and various techniques have been developed to address UAV exploration. *Sampling-based* and *Frontier-based* approaches have been extensively studied, and recent efforts focus on combining the strengths of both approaches to enhance performance. Additionally, advancements have been made in integrating advanced solvers and collaborative exploration strategies to optimize exploration time and improve efficiency.

3. Methodology

This section aims to provide a detailed description of the UAV Exploration Framework, highlighting its general components and the specific focus of the project. The framework serves as the foundation for autonomous exploration and offers insights into the systematic exploration of indoor environments.

The section begins by introducing the UAV Exploration Framework, emphasizing its key elements and overall structure. It then delves into the Frontier-based exploration approach, which forms a fundamental component of the framework. The process starts with frontier detection to identify unexplored regions, followed by clustering techniques to group frontiers based on proximity and similarity. The section concludes by describing the selection process, where the most promising frontiers are chosen as exploration targets.

Furthermore, the section describes the complementary systems integrated into the framework. These include mapping, planning, navigation, and TSP solvers, all of which play crucial roles in enabling an effective exploration.

3.1. UAV Exploration Framework

The Autonomous Exploration Framework encompasses all the necessary systems for the proper functioning of a UAV. As is showcased in Fig. 3.1, it begins with the Localization system, which provides accurate estimation of the UAV's pose description in a global frame of reference. Following this, the mapping system is responsible for systematically storing the gathered environmental information, enabling the definition of obstacle locations, shape, and free spaces, thereby determining the navigable areas for the UAV.

The Exploration system utilizes the aforementioned map structure to identify potential targets that need to be visited in order to fully cover the desired environment. The generic process for Frontier-based exploration using Octomap as mapping Framework is illustrated in Figure 3.2.

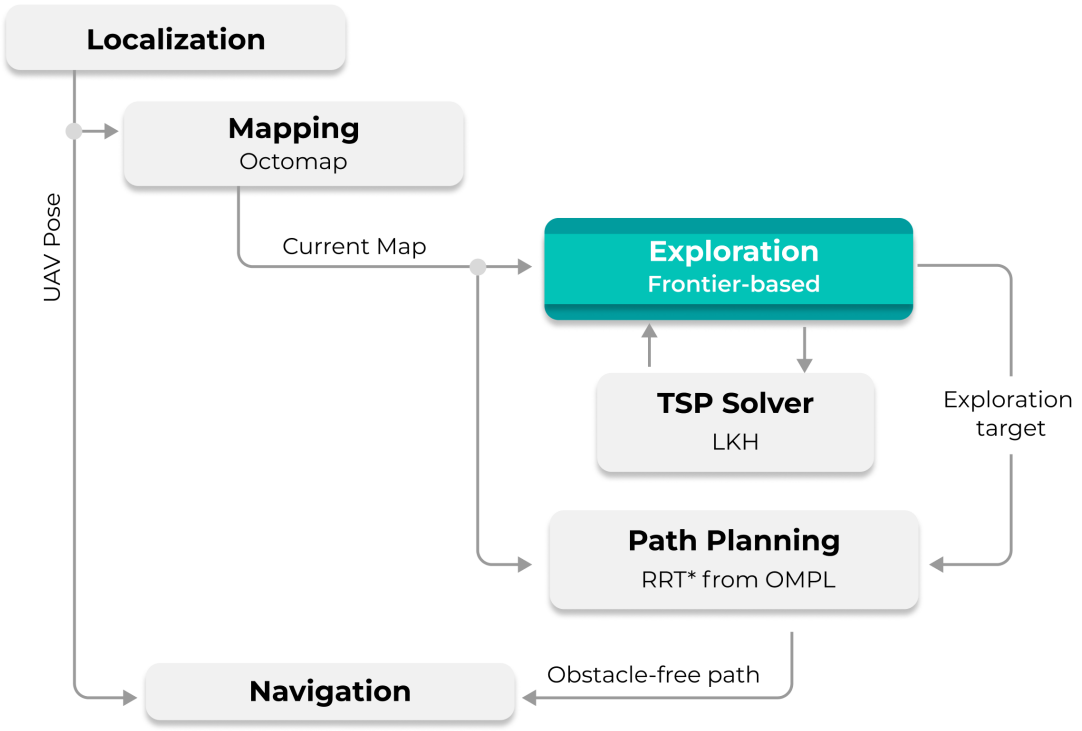


Figure 3.1: General Autonomous Frontier-based Exploration Framework

This process involves analyzing a 3D occupancy map to identify frontier nodes, which are located at the boundaries between free and unknown spaces. For this purpose, a set of neighbors for each analyzed node is computed. Within this set, the occupancy state of each neighbor is verified to identify the nodes that meet the criteria for being classified as frontier nodes.

Once the frontier nodes are detected, a clusterization phase groups proximate frontier nodes into clusters. Through the analysis of these clusters and a utility function, the next exploration target is determined and passed to the Path Planning algorithm. This algorithm generates an obstacle-free trajectory based on the mapping system's information, facilitating the UAV's navigation towards the target.

Once a safe trajectory has been generated, the Navigation system comes into play, generating control commands to guide the UAV accurately along the predefined trajectory. The exploration process is iterative, repeating until no frontiers remain to be explored. The integration of these interconnected systems ensures efficient and comprehensive mapping of the environment by the UAV.

Lastly, it is important to highlight the dependency of all these systems on the UAV's sensor system. In addition to the sensors responsible for navigation and

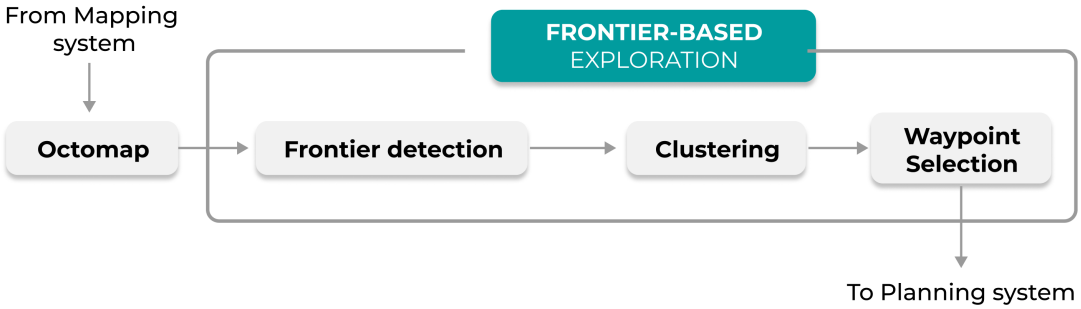


Figure 3.2: Frontier nodes detection and selection based on Octomap

basic vehicle operation, the proposed exploration algorithm specifically utilizes a lidar scanner as the primary input for the mapping phase, forming the foundation for subsequent processes.

3.2. Frontier-based Exploration

3.2.1. Proposed approach: Collector Exploration

Greedy exploration strategies follows the generic process for frontier detection and selection shown in 3.2. This approach can be tailored to minimize travel distance by targeting the nearest exploration point, or to maximize information gathering by considering sensor range and model. The former approach often results in a slower exploration, while the latter leads to faster, but potentially incomplete, exploration.

Within this context, a strategy designated in this project as the ”Collector” is implemented. This strategy incorporates two extra stages - storage and filtering - to validate the clusters previously detected through the Frontier-based exploration framework, as depicted in Figure 3.3. First, frontiers are identified and clustered similarly to the process in greedy exploration. Second, all detected clusters at any stage of the exploration are stored in an array. This array undergoes filtering at each iteration to eliminate obstructed or similar points, previously visited locations, or targets offering low information gain. Ultimately, in the third stage, the nearest cluster is chosen as the next exploration target and forwarded to the path planning module.

Furthermore, building upon the work conducted by Zhou et al. (2021), this study also assesses the feasibility of incorporating a solution to the Travelling Salesman Problem (TSP). The purpose of this solver is to determine the most

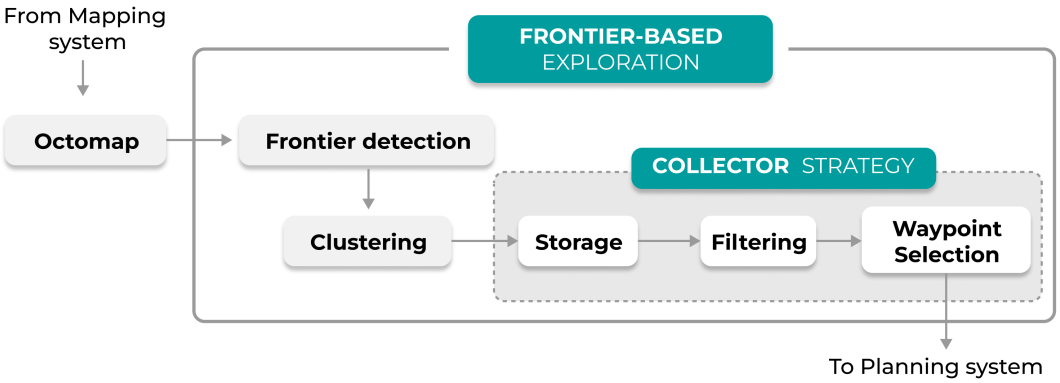


Figure 3.3: Collector exploration strategy process

efficient route to navigate through all the exploration targets contained in the exploration list. Below is the description of the used mapping system.

3.2.2. Mapping

In this work, the Octomap Framework, developed by [9] is used as mapping system. OctoMap is a 3D occupancy mapping framework widely used in robotics and autonomous systems. It provides a probabilistic representation of the environment by dividing the space into small volumetric elements called voxels. Each voxel, also known as node, stores information about its occupancy state, indicating whether it is free, occupied, or unknown. OctoMap efficiently maintains and updates the occupancy information as the robot explores the environment. OctoMap's probabilistic nature allows for uncertainty modeling, enabling the representation of both known and unknown areas with different levels of confidence.

Furthermore, OctoMap includes convenient iterators which streamline the search and exploration of particular nodes within the map. These iterators enable efficient navigation of the octree data structure, granting easy access to individual or grouped voxels based on specific user criteria. This feature is particularly advantageous in frontier exploration, as it aids in identifying nodes located at the frontiers of the current map. The KeySet data structure is prevalently employed in the algorithm, given its utility in simultaneously accessing and manipulating multiple keys. It also allows efficient iteration over the keys, facilitating operations such as insertion, removal, and intersection. Within the Octomap framework, a Key serves as a unique identifier representing a specific node or leaf node in the octree data structure, and it provides a compact portrayal of a node's position

within the octree hierarchy.

However, in scenarios involving large-scale 3D exploration, the search for frontiers in the global map becomes computationally challenging due to the vast number of nodes created. To address this issue, OctoMap provides a specialized method that iterates over the modified nodes, streamlining the handling of changes in the map's structure. This functionality allows for targeted processing, focusing only on the regions that have undergone modifications since the previous change detection call. By disregarding unchanged areas, this approach significantly reduces the computational demand associated with frontier exploration in OctoMap.

Overall, OctoMap offers a powerful solution for maintaining a detailed and probabilistic representation of the environment, making it a valuable tool for various robotic applications that require 3D mapping and understanding of the surroundings.

3.2.3. Frontier detection

The frontier detection serves as the initial stage of the proposed exploration framework, primarily accomplished by analyzing the Octree structure and leveraging the available iterators within the Octomap library. This process is carried out by using the frontier detection algorithm (3D-FBET) presented by [21] where the detection of frontiers is restricted to the nodes of the Octree that have been modified since the Octomap generator's last invocation, as depicted in Algorithm. 1. Additionally, Figure 3.4 provides a visual depiction of the frontier nodes and the ensuing clustering process, which will be explicated in the subsequent section.

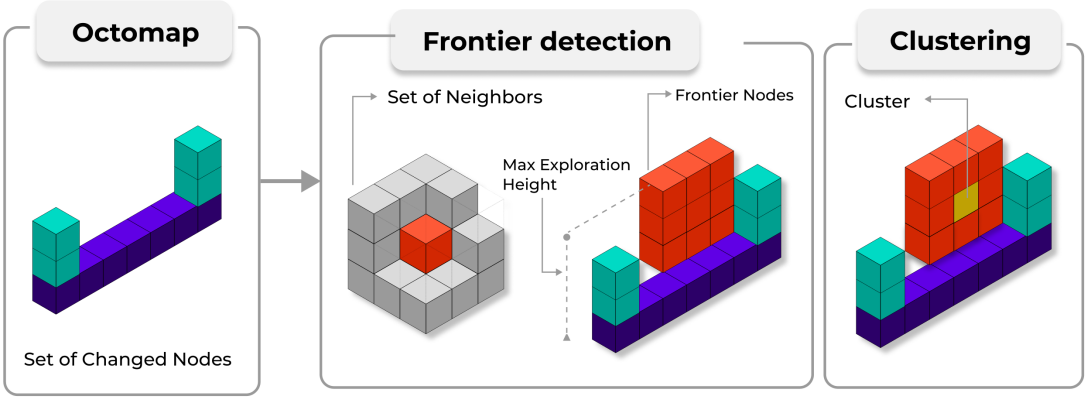


Figure 3.4: Collector exploration strategy process

This algorithm processes the set of nodes in the octree that have been changed in the current iteration and examine each node individually.

For every node within predefined exploration limits, the algorithm assesses its occupancy status. If the node isn't occupied, the algorithm creates a set of the node's twenty-six neighboring nodes. Subsequently, the algorithm traverses each neighbor.

During this traversal, when an unknown space (null neighbor) is found, the unknown space flag is set to true. Similarly, when an unoccupied neighbor is found, the free space flag is set to true.

At the conclusion of this process, if both the unknown space and free space flags are true, the node is considered a frontier node. This denotes that the node is at the boundary between known and unknown spaces. All such frontier nodes are collected for further processing.

Algorithm 1: Frontier detection

Input: *changedNodes*: the nodes of the octree that have been modified in the current iteration

Output: *frontierNodes*: the octree nodes located in the boundaries between unknown and occupied spaces

```
unknownNodeFlag ← false;
freeNodeFlag ← false;
foreach node in ChangedNodes do
    if node is inside exploration limits then
        changedCellOcuppied = isNodeOccupied(cell);
        if ¬changedNodeOccupied then
            unknownNodeFlag = false;
            freeNodeFlag = false;
            genNeighborCord(node, neighborNodes);;
            foreach neighbor in neighborNodes do
                if neighbor == NULL then
                    unknownNodeFlag = true;
                else if ¬isNodeOccupied(neighbor) then
                    freeNodeFlag = true;
            if unknownNodeFlag ∧ freeNodeFlag then
                frontierNodes.insert(node);
```

3.2.4. Frontiers analysis

This section delves into the additional phases of storing and filtering as delineated within the Collector strategy. Figure 3.5 provides a comprehensive visual representation of these processes.

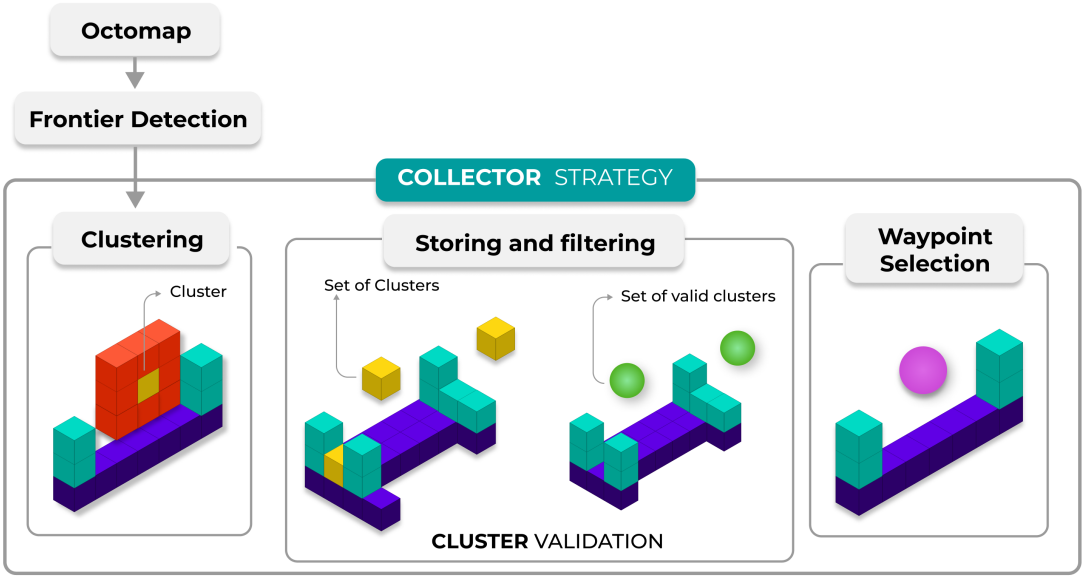


Figure 3.5: Collector exploration strategy process

Clustering

Once the frontier nodes have been identified and stored in the corresponding KeySet, a clustering process is initiated using the Mean-Shift Clustering algorithm. The Mean-Shift Clustering algorithm introduced by [6] is an iterative method that identifies dense regions of data points in a three-dimensional space by shifting a window towards regions with higher data density. Typically, a cluster node correspond to the centroid of a group of frontier nodes as depicted in 3.4.

These clusters represent potential exploration targets for the UAV. Specifically for this work, an adaptation of this algorithm, designed to work with Octomap nodes and developed by [1], is employed.

Clusters validation

In the proposed method, all detected clusters throughout the exploration process are retained in an array. Depending on the chosen cluster bandwidth, this array may contain clusters with similar positions.

As the exploration progresses and the UAV uncovers new areas of the map, certain clusters may become obstructed by obstacles, rendering them invalid as waypoints. Furthermore, some clusters may reside in already explored zones or in regions offering minimal information gain.

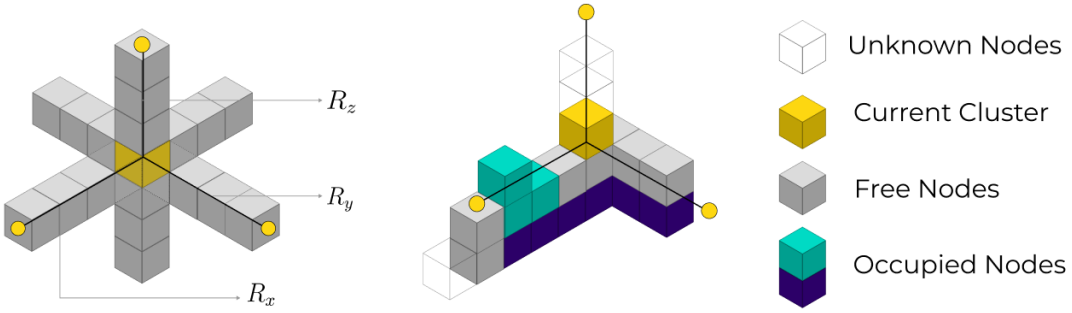


Figure 3.6: Representation of information gain domain set by six equidistant points

This issue is tackled by preserving only the clusters that possess an information gain exceeding a predefined threshold, one of the method's hyperparameters. Accordingly, to eliminate similar, obstructed, or unproductive nodes, it becomes necessary to update the list of clusters, thereby generating a renewed set of potential exploration waypoints. This is accomplished using Algorithm 2.

This algorithm takes an array of detected clusters as input and proceeds to compute the information gain for each. The information gain for each cluster is computed considering only in the nodes that lies in the domain generated by the set of six equidistant points distributed along the three spatial axes with respect to the current cluster position as illustrated in Figure 3.6. Upon completion of the nested loops, the algorithm calculates the information gain as the ratio of unknown nodes to total nodes. This quantifies the extent of unexplored space, thus offering a measure of information gain.

Subsequent to the computation, the algorithm evaluates the information gain relative to a pre-set threshold. In addition to this, validity checks are performed on the clusters to identify those that have been previously assigned or have become obstructed. These validity checkers are depicted in Figure 3.7.

The determination of whether a cluster has become obstructed involves the analysis of a cube centered on the cluster position, as shown in Figure 3.7a. The dimensions of the cube (l) are matched with the state validity checker incorporated within the planner module to prevent potential planning failures due to invalid starting or ending points. The cube is sampled along the three axes using the predefined octomap resolution. Should any of the nodes within the cube

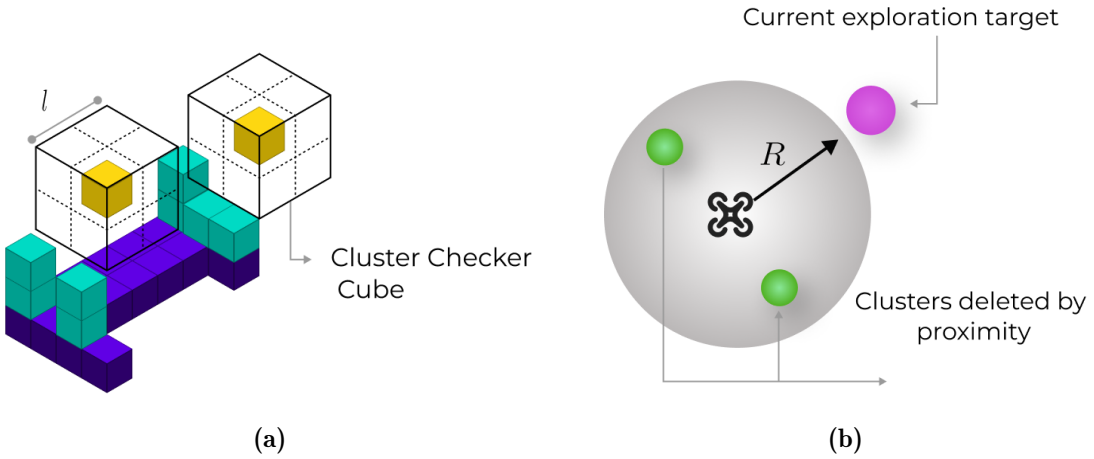


Figure 3.7: Implemented state validity checkers - a) Obstruction checker, b) Similarity checker

be identified as occupied, the cluster is deemed invalid and removed from the updated list of potential exploration targets.

Furthermore, the algorithm examines a historical list of previously visited or assigned clusters. This is achieved by analyzing a sphere of radius R centered on the UAV's current position, allowing the algorithm to exclude points that the UAV encounters within a specified exploration radius while navigating towards another point. Figure 3.7b represents this verification process. Such scenarios might emerge owing to the sampling nature of the RRT* planner strategy, a topic that will be expanded upon in the following section.

If a given cluster has passed all the checkers, the algorithm inserts the cluster into the potential exploration target list ($allClustersUpdated$). This signifies that the cluster provides sufficient information gain and meets all necessary conditions to be included in the updated set of clusters.

In summary, the Clusters Update algorithm effectively refines the set of detected clusters, prioritizing those that offer significant information gain and fulfill necessary conditions for further exploration.

Exploration target selection

The Cluster Selection algorithm, as depicted in Algorithm 3, is essentially an iterative minimization process that identifies the cluster nearest to the UAV's current position. It operates on an array of validated clusters.

The algorithm examines each cluster in the array in turn. For every cluster, it calculates the Euclidean distance between the UAV's present position and that

Algorithm 2: Clusters Update

Input: *allclusters*: clusters detected in the current iteration

Output: *allClustersUpdated*: updated array of clusters based on information gain

Let *allClustersUpdated* be the array of detected clusters;

Let *cubeChecker* be an array of six equidistant points distributed in x, y, and z axes (positive and negative) considering the current cluster position as the origin;

```
foreach cluster in allClusters do
    allCounter ← 0;
    unknownCounter ← 0;
    infGain ← 0;
    foreach cube_point in cubeChecker do
        computeRayKeys(cluster, cubeChecker);
        foreach Key in KeyRay do
            allCounter++;
            if Key == NULL then
                allUnknown++;
    infGain = unknownCounter/allCounter;
    if infGain < infThreshold ∨ isClusterAlreadyAsigned ∨
     isPointAlreadyAsigned ∨ isClusterObstructed then
        continue;
        else
            allClustersUpdate.insert(cluster)
```

of the cluster. If this computed distance is smaller than the current minimum, it updates the minimum distance.

If, at the end of the process, no suitable cluster has been identified, the algorithm flags the absence of a feasible frontier. Otherwise, the algorithm selects the cluster corresponding to the minimum distance as the most desirable one. Upon completion, the algorithm provides the most desirable cluster as output, representing the cluster nearest to the UAV's current position.

Algorithm 3: Cluster Selection

Input: *current_pose*: UAV current position

Output: *best_cluster*: nearest cluster from UAV current position

```
noBestFrontier ← false;
min ← ∞;
Let allClustersUpdated be the array of detected clusters;
foreach cluster in allClustersUpdated do
    dist = euclideanDistance(current_pose, cluster);
    if dist < min then
        min = dist;
        if min == ∞ then
            noBestFrontier=true;
        else
            best_cluster=cluster;
```

In the next section, the Path Planning system is further described.

3.2.5. Path planning

The exploration framework's path planning stage incorporates the RRT* (Rapidly-exploring Random Tree Star) algorithm, which is an extension of the original RRT algorithm designed to overcome some of its limitations. RRT* improves the optimality and convergence properties of generated paths by performing rewiring operations during the expansion phase. Unlike RRT, which only connects new samples to their nearest neighbors, RRT* considers rewiring connections to nearby configurations that may result in more optimal paths. This adaptive rewiring step allows the tree to continuously adapt and refine the paths as more iterations are performed, ultimately converging towards near-optimal solutions.

An essential component of the planning system is the state validity checker, which evaluates the validity of configurations based on system constraints, such as collision avoidance and environment-specific criteria. During the expansion of the RRT tree, when new configurations are sampled or interpolated between existing configurations, the state validity checker is invoked to assess their validity. It checks for constraint violations and collisions with obstacles in the environment, ensuring that the generated paths are collision-free and compliant with system limitations.

The RRT* algorithm has been extensively studied and proven effective in various motion planning tasks. In this work, the implementation utilizes the RRT* planner available in the OMPL library, as presented in the benchmarking study by [12]. Additionally, a specialized adaptation of the RRT* algorithm specifically tailored for UAV applications, developed by [10], is employed.

In practical implementation, the path planner takes a binary representation of the Octomap as input, providing an efficient and compact description of the environment. A prism-like shape state validity checker (SVC) is used as a simplified representation of the UAV.

Once the inputs are determined, the path planner formulates a trajectory comprising a sequence of points defined by their spatial and angular coordinates (x, y, z, ψ). This trajectory serves as the guiding path for the Navigation module.

In addition to this, an extra validity checker is implemented to identify instances where the current path may have become invalid. Such a situation can occur when newly discovered obstacles appear in or near the path. This validity checker uses a method similar to the frontier detection to analyze the surroundings of a given point, as depicted in Figure 3.8.

If the validity checker detects an obstruction in the path during exploration, the UAV is halted, and a new path is requested, maintaining the same destination. This ensures the UAV doesn't attempt to traverse through newly discovered obstacles, thereby enhancing the safety and efficiency of the exploration.

3.2.6. TSP Solver

As noted at the beginning of this section, this study also evaluates the usability and effectiveness of incorporating a Travelling Salesman Problem (TSP) Solver to enhance exploration speed and quality. To this end, the well-known Lin-Kernighan-Helsgaun (LKH) solver is employed, an heuristic algorithm formulated

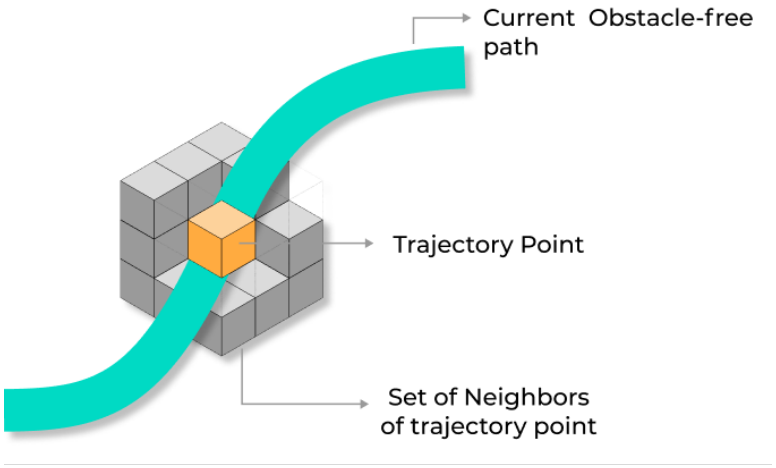


Figure 3.8: Validity checker for current path

by [7].

The LKH solver operates on the principle of iterative improvement, wherein it continuously tries to find better routes by disconnecting and reconnecting the path between nodes in different ways to minimise the total distance travelled. It is noteworthy that the algorithm does not guarantee the absolute shortest path; rather, it seeks to find a route that is as close as possible to the optimal one given its constraints.

The solver takes as input the set of checked cluster nodes. From this set, a cost matrix, as outlined in Equation 3.1 is computed. This matrix embodies the displacement cost incurred when moving between any two clusters within a given list with n points. In the context of this research, the Euclidean distance was chosen to approximate the cost of travel between two clusters. This distance must be computed for all possible combinations to populate the cost matrix accurately.

Furthermore, the cost matrix can be structured to signify either closed or open-loop problems. In this study, an open-loop TSP solution is desired from the solver, achieved by initializing the first column of the cost matrix to zero.

$$C_M = \begin{bmatrix} d_{00} & d_{01} & d_{02} & \dots & d_{0n} \\ d_{10} & d_{11} & d_{12} & \dots & d_{1n} \\ d_{20} & d_{21} & d_{22} & \dots & d_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ d_{n0} & d_{n1} & d_{n2} & \dots & d_{nn} \end{bmatrix} \tag{3.1}$$

Despite the fact that a straight-line distance may not always correspond to

the shortest path—particularly in environments with obstacles—its computation remains simple and efficient. Nonetheless, the primary aim of this stage is to evaluate the utility of this method, as well as ascertain its effectiveness in enhancing the speed and quality of the exploration. In practice, as seen in Figure 3.1, the TSP Solver module calculates the optimal route through the exploration targets previously identified by the frontier exploration module. Additionally, to gauge the impact of incorporating this module into the exploration framework, a hyperparameter has been established. This hyperparameter sets the number of clusters within the tour that must be visited before a new TSP route is solicited.

After this detailed overview of the methodology and applied techniques, the next section outlines the system's technical implementation within the ROS Framework.

3.3. ROS Architecture

This thesis project leverages a ROS ecosystem, taking advantage of its modularity and efficiency in managing sensor data and system interactions. To facilitate development, and testing, the system has been implemented within two modular containers. This approach aids in overcoming dependency incompatibilities and simplifies system maintenance. As illustrated in Figure 3.9, two container systems are utilized. The first is a Singularity container from the Multi-robot Systems Group (MRS) at the Czech Technical University in Prague [5]. Notably, this Singularity container comprises a control, estimation, and simulation platform encapsulating a complete suite of subsystems associated with UAV operation. Importantly, it includes the full operating system of the quadrotor UAV used in subsequent flight testing stages. Secondly, a container provided by the Laboratory for Robotics and Intelligent Control Systems at the University of Zagreb, [1] was used as a reference framework and base environment for the development and implementation for the this project.

Figure 3.10 provides an overview of the system, accentuating the implemented nodes, and the key topics, subscribers, publishers, and services within the system. The system's primary data source is a 3D LiDAR scanner that supplies range measurements in the form of point cloud messages. Further, odometry estimates from the localization system provide a reference frame for the generated OctoMap and subsequent frontiers and exploration targets.

The **frontmanager** node is at the core of the system, responsible for generat-

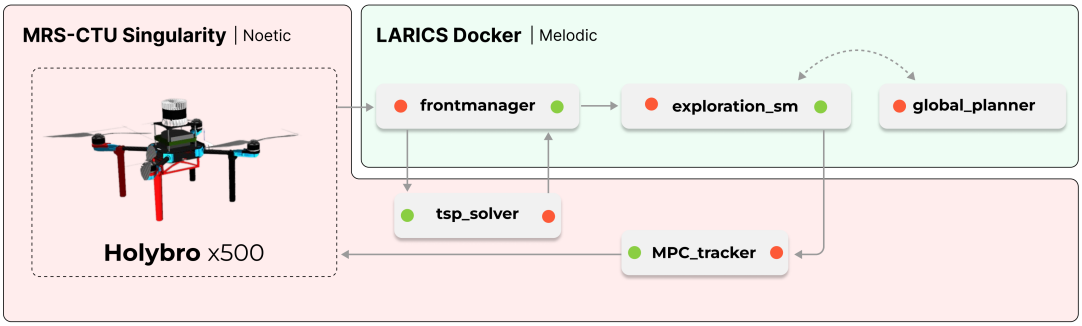


Figure 3.9: Autonomous Frontier-based exploration for UAV Framework

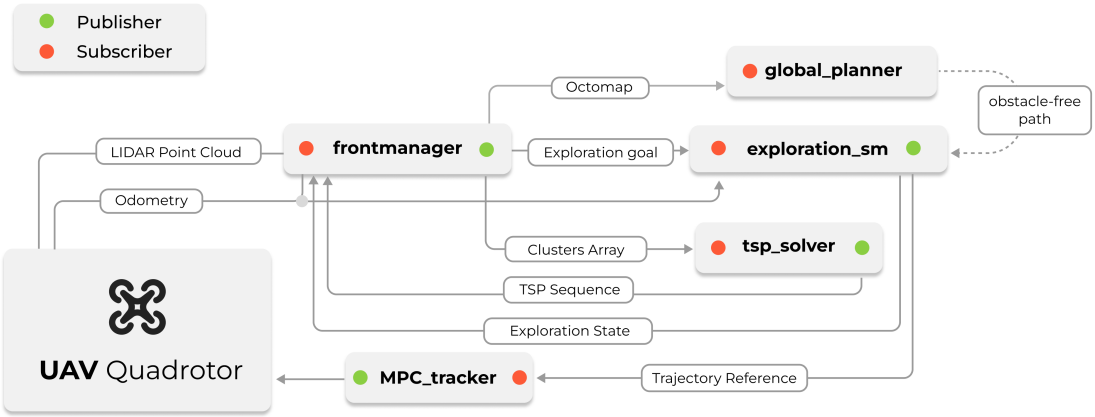


Figure 3.10: Autonomous Frontier-based exploration for UAV Framework

ing the OctoMap and executing the frontier detection, clustering, updating, and selection algorithms. This node generates and publishes a binary OctoMap and the exploration goal as a Pose message. It is worth noting that this implementation does not employ the default OctoMap Server from the OctoMap Library. Instead, all required methods for generating the OctoMap tree and its nodes from the provided point cloud are incorporated within the **frontmanager**.

The **global_planner**, operating in a separate node, serves as the path planner module. It is invoked through a service and requires the UAV's current and target pose as input.

Another developed node, the **exploration_sm**, functions as a State Machine managing planner calls, responses, and exploration stages. It interprets the trajectory response from the **global_planner**, converting it to a Trajectory Reference message type [4]. This step is important for integration with the broader UAV manufacturer's ecosystem. Finally, the trajectory message is forwarded to the **MPC_tracker** node, which is responsible for executing the final control commands.

A comprehensive ROS architecture diagram is included in the appendix section for more detail.

4. Results

The current project has undergone rigorous testing in a simulation environment utilizing the MRS Group's platform from the Czech Technical University as a foundational system. Within this simulation platform, three distinct exploration strategies - Greedy, Collector, and Multi-resolution - were assessed across four different maps. This section begins with an evaluation of the potential benefits of incorporating a Travelling Salesman Problem (TSP) Solver to enhance overall exploration performance. Subsequently, the exploration strategies are examined across warehouse-like environments of varying complexity. This evaluation allows for a more nuanced understanding of the algorithm's performance under differing conditions. Finally, this section concludes with the presentation of preliminary results from initial flight tests. These crucial tests served to confirm the proper functioning of the entire system when operational aboard a UAV Quadcopter.

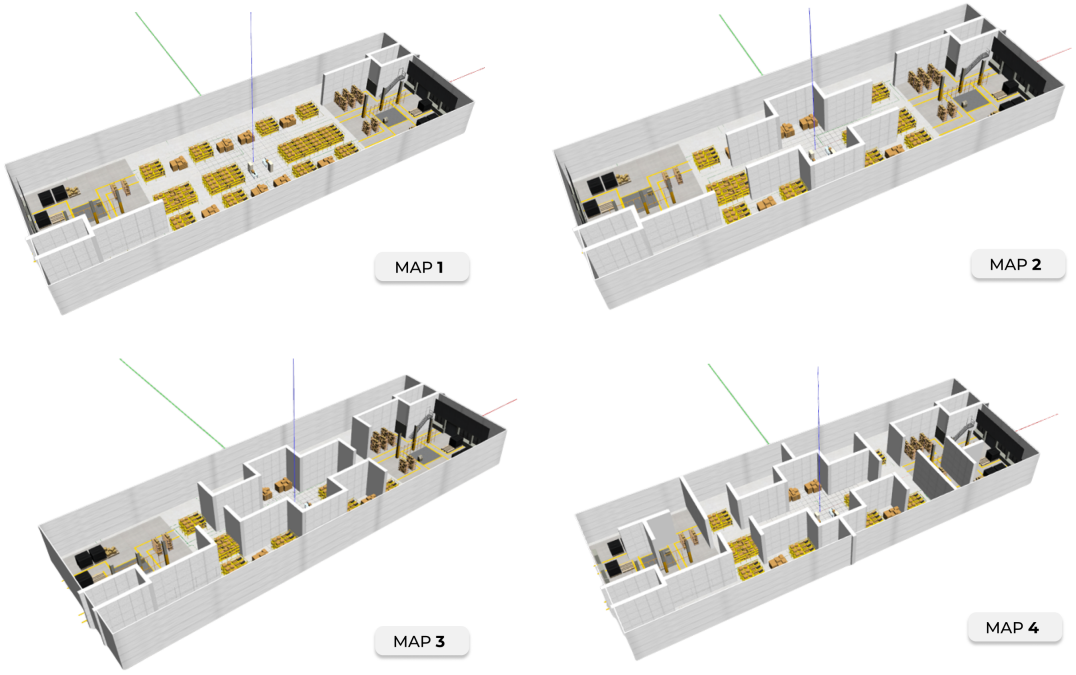


Figure 4.1: Warehouse-like maps used in simulation testing

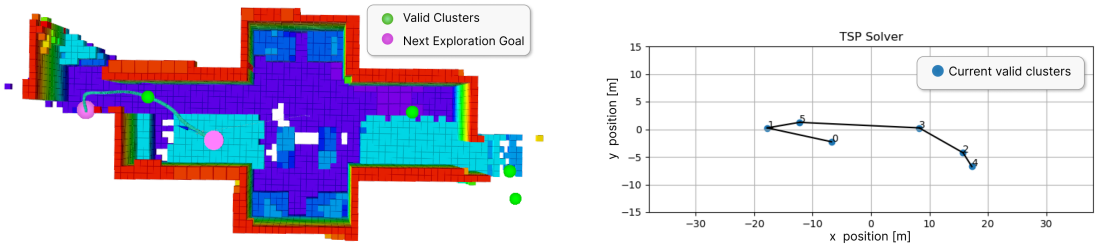
In order to accurately assess the developed algorithm and the comparison approaches, four simulated warehouse-like environments were created. Each of these environments, shown in Figure 4.1, represents varying levels of complexity. Map 1, for example, is the least complex, offering substantial open spaces. In contrast, Map 4 is the most intricate, characterized by a considerable number of walls and hidden spaces that the UAV must navigate during exploration. By progressing through environments of increasing complexity, it becomes possible to effectively evaluate the exploration algorithms in a variety of scenarios, providing a more comprehensive understanding of their respective strengths and limitations.

4.1. TSP Solver Influence

A TSP solver strives to find the most efficient route to traverse a predefined set of locations. In the context of autonomous exploration, the TSP is deployed to discern the optimal route for visiting a collection of exploration targets. This approach particularly enhances non-greedy strategies like the proposed "Collector" method, wherein potential targets are accumulated in a list for systematic analysis, irrespective of their individual information value. Given the dynamic nature of potential target sets, a hyperparameter has been introduced that dictates the number of points the UAV should visit before a fresh TSP solution is requested. This strategy prevents continuous solver calls and facilitates a clearer evaluation of its advantages and limitations.

Figure 4.2a illustrates an instance of this algorithm operating with the TSP Solver during a collector exploration. The UAV is shown en route to the first point on the TSP tour. Upon completion of the of set of predefined tour points, a new TSP solver request is submitted. Consequently, a tour akin to the one depicted in 4.2b is charted. It is worth noting that the TSP tour is computed for the entire list of valid clusters available, regardless of the predefined TSP tour parameter.

Table 4.1 details the results for Exploration Time (Exp.Time), Distance Traveled (Dist.T), and percentage of Explored Map (Exp.Map). Exploration Time signifies the time taken until the algorithm determines there are no more frontiers to visit. The Distance Traveled metric is derived from the odometry estimation obtained from the simulation platform. Explored Map, indicative of the percentage of the map explored, is computed by assessing the length of the Octomap Message generated by the Octomap. It is presupposed that a larger map message



(a) (b)

Figure 4.2: Collector exploration running along with TSP Solver - a) RVIZ visualization b) Current TSP Tour

implies a greater volume of data gathered by the UAV. For this final metric, the maximum value obtained across all tests served as a benchmark for evaluating the relative percentages of the other tests.

The table showcases results for different quantities of Tour Points. The data was obtained by implementing the Collector Exploration Strategy on Map 1 and Map 3, as shown earlier in Figure 4.1. Furthermore, the cost matrix for the TSP solver was constructed by determining the Euclidean distance between the desired locations.

Importantly, when the number of requested Tour Points surpasses the number of available potential targets, the exploration algorithm will adjust the Tour Points parameter to the maximum possible.

Tour P.	**Map 1**			**Map 3**		
	Exp. Time	**Dist. T [m]**	**Exp.Map**	**Exp. Time**	**Dist. T[m]**	**Exp.Map**
1	5m38s	211.01	92.9%	6m42s	239.48	96.54%
3	6m01s	222.91	94.5%	8m57s	359.45	100%
5	9m01s	375.29	100%	10m52s	401.18	98%
7	8m16s	339.51	94.7%	11m07s	454.1	92.2%
9	6m45s	268.71	94.5%	9m97s	380.86	94.1%

Table 4.1: TSP Assessment

As seen in Table 4.1, the general trend indicates that an increased number of tour points corresponds with greater travel distance and exploration time, which is not desirable for exploration algorithms. Concerning the information gathered, the percentages appear equivalent as no significant disparities are evident. The exploration time and distance increments are most noticeable in Map 3. This

analysis indicates that the TSP Solver, in its current implementation, did not contribute any benefits in these tests. The principal limitation being that Euclidean distance isn't an effective cost function, especially in environments with obstacles. As a result of this evaluation, and considering that setting Tour Points to 1 yielded the best results in both maps. Consequently, it was determined that the Collector method would integrate an exploration policy always aiming for the nearest exploration point. This decision was influenced by the observation made during the tests that the first TSP point is typically the closest one.

4.2. Warehouse Environments

Utilizing the previously described environments, two additional exploration strategies, beyond the proposed Collector approach, were assessed to better understand the developed algorithm's performance.

The first approach assessed is a Greedy strategy, wherein the cluster possessing the highest information gain, computed using the process described in *Frontier Analysis section*, is selected as the exploration target during each iteration.

The second approach corresponds to a state-of-the-art algorithm developed by Batinovic et al. (2021). This innovative method leverages the multi-resolution capabilities of the Octree data structure to perform a more efficient frontier search. Furthermore, the algorithm introduces an enhanced information gain function that evaluates potential targets within a condensed volume, thereby enhancing computational efficiency.

The results of an exhaustive evaluation of these three distinct algorithms across four different maps (M1 to M4), are succinctly summarized in Table 4.2. The evaluation of each algorithm was performed by assessing three parameters: exploration time, distance traveled, and the proportion of the map explored. These parameters offer a comprehensive measure of each algorithm's efficiency and effectiveness in different exploration contexts, providing valuable insights into their respective performance characteristics. Notably, for the Greedy and Multi-resolution methods, the exploration is deemed complete when no further clusters are detectable. Conversely, with the Collector approach, the exploration concludes when no more checked clusters persist in the list of potential exploration targets.

The Greedy approach, despite achieving close to or exactly 100% exploration across all maps, specially in less complex maps (M1 and M2), displayed a tendency

for longer exploration times and greater traveled distances. This was particularly apparent on the complex Map M4, where it registered an average exploration time of 11.59 minutes and an average distance of 508.23 meters. The nature of this algorithm often leads to considerable back-and-forth movement, which can result in a less efficient exploration trajectory.

The Multi-resolution algorithm typically finished exploration in the shortest average time and covered the smallest distances across all maps. However, it somewhat fell short in terms of exploration completeness, especially on the more complex M3 and M4 maps, when compared to the other two approaches.

Amid these varying results, the Collector algorithm showcased a consistent and adaptable performance across all maps, even in face of escalating complexity. It efficiently balanced exploration time, distance covered, and exploration completeness. For instance, on Map M4, the Collector algorithm took an average exploration time of 9.43 minutes, which is shorter than the Greedy approach, and traveled a shorter distance of 383 meters, while still achieving a 97.96% exploration rate. Similarly, on the challenging environment of Map M3, it achieved an exploration rate of 97.15%, higher than the Multi-resolution approach's 94.74%, while maintaining a more efficient exploration time and traveled distance.

Overall, in the challenging environments of Maps M3 and M4, the Collector algorithm not only outperformed the Multi-resolution approach in terms of map coverage, but also took less time and covered shorter distances than the Greedy approach. This attests to the algorithm's ability to maintain an effective balance of thoroughness and efficiency.

In conclusion, the choice of exploration algorithm depends largely on the specific requirements and objectives of an application. Each algorithm, whether it's the Greedy, Multi-resolution, or the Collector, presents its own advantages in certain aspects of the exploration task. The Collector algorithm, particularly, exhibits an encouraging balance of speed and comprehensiveness, especially on more complex maps, highlighting its suitability for applications where both efficient and thorough exploration is essential. Ultimately, the selection of an exploration strategy should be tailored to emphasize the criteria that hold the most importance for the given scenario or application.

Having conducted a thorough analysis of the significant insights concerning the performance of the various exploration strategies across the warehouse maps, the subsequent figures are designed to illustrate the typical behavior of the tested strategies, specifically in relation to exploration trajectories, traveled distance,

Greedy									
Map	**Exploration Time [min]**			**Distance Traveled [m]**			**Explored [%]**		
	Avg	**Max**	**Min**	**Avg**	**Max**	**Min**	**Avg**	**Max**	**Min**
M1	5.38	6.08	4.38	237.2	272.42	195.12	96.85	100	94.35
M2	8.77	10.13	6.88	373.48	454.14	283.4	98.19	100	94.99
M3	7.9	10.13	5.18	337.35	440.25	218.52	93.98	97.22	91.13
M4	11.59	16.7	7.83	508.23	686.94	354.08	96.33	99.19	92.42
Collector									
Map	**Exploration Time [min]**			**Distance Traveled [m]**			**Explored [%]**		
	Avg	**Max**	**Min**	**Avg**	**Max**	**Min**	**Avg**	**Max**	**Min**
M1	6.48	6.67	6.3	280.17	288.8	271.4	96.14	99.32	92.95
M2	8.76	9.23	7.87	354.5	380.19	311.7	97.73	99.05	95.83
M3	7.67	10.72	5.3	312.49	390	219.92	97.15	100	93.43
M4	9.43	10.87	7.5	383	459.24	318.23	97.96	100	93.28
Multi-resolution									
Map	**Exploration Time [min]**			**Distance Traveled [m]**			**Explored [%]**		
	Avg	**Max**	**Min**	**Avg**	**Max**	**Min**	**Avg**	**Max**	**Min**
M1	3.45	3.54	3.36	162.2	167.8	156.64	87.92	90.36	85.48
M2	5.36	5.42	5.31	218.6	222.88	214.36	91.76	94.04	89.47
M3	6.77	9.15	4.4	284.38	386.97	181.8	94.74	95.87	93.62
M4	6.28	6.29	6.27	266.59	270.87	262.31	92.8	93.91	91.69

Table 4.2: TSP Assessment

and the percentage of completed map exploration.

Initially, the performance of the algorithms on Map 1 is depicted in Figures 4.4 and 4.3 In terms of exploration trajectories, it is evident that the Greedy and Multi-resolution strategies tend to generate long paths which usually traverse a great portion of the environment, while the Collector strategy executes a more meticulous exploration, as further demonstrated by the higher number of visited exploration targets. Regarding the percentage of explored map, the Collector strategy takes the longest time to complete the map. On the other hand, the Multi-resolution strategy accounted for an average of 87.92% map exploration, marking the lowest level of map coverage among the tested approaches. However, the Greedy strategy demonstrates an optimal balance between the duration of exploration and map completion, indicating its superior efficiency.

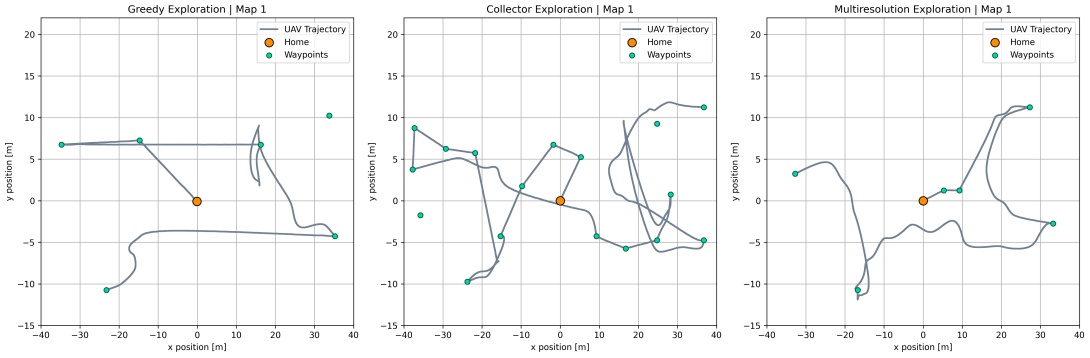


Figure 4.3: Exploration trajectories on Map 1

However, the aforementioned scenario does not pose a significant challenge to exploration algorithms. Therefore, to gain more insightful observations about the behavior and performance of the algorithms, the more complex Map 4 is analyzed next. Figure 4.5 provides a clear illustration of the distinguishing characteristics of the evaluated approaches. The Greedy strategy is marked by back-and-forth movements, while the Multi-resolution strategy, despite having fewer redundant paths, neglects to visit significant portions of the map. Conversely, the Collector strategy strikes a better balance between these extremes by having relatively fewer redundant paths and visiting more exploration targets. As a result, it completes the map in less time and distance than the other approaches, as illustrated in Figure 4.6.

Upon evaluating the proposed Collector approach and its comparative strategies in the warehouse-like environments, the subsequent section seeks to demonstrate the performance of the Collector strategy within an environment that per-

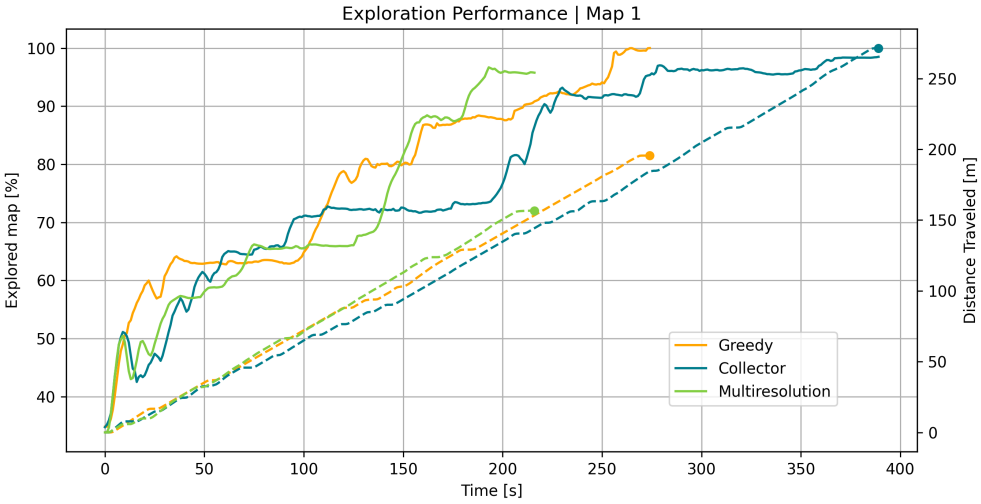


Figure 4.4: Explored map percentage (non-dashed) and distance traveled (dashed) comparison on Map 1.

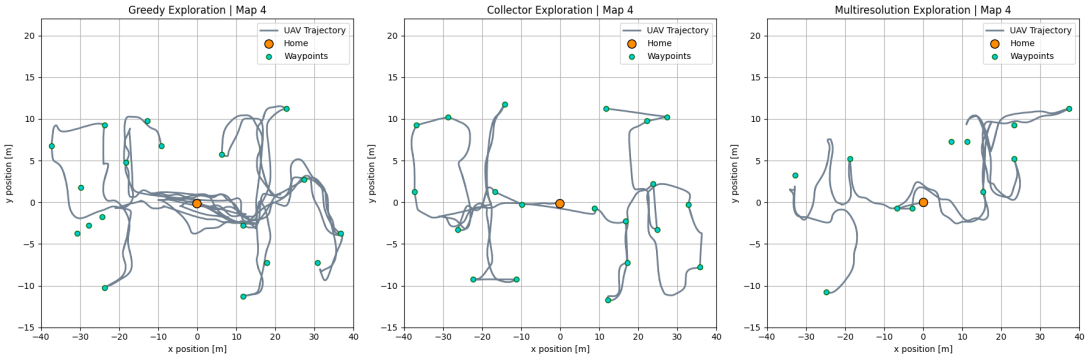


Figure 4.5: Exploration trajectories on Map 4

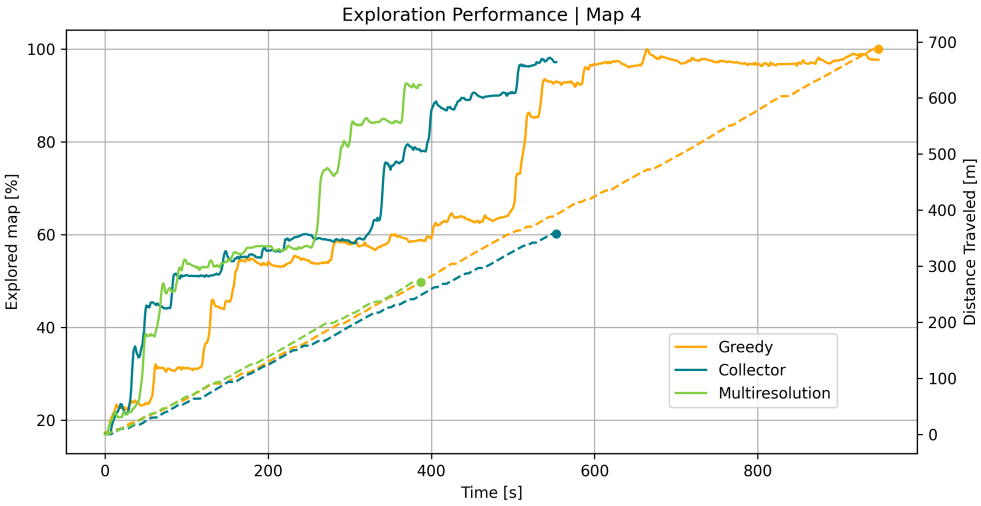
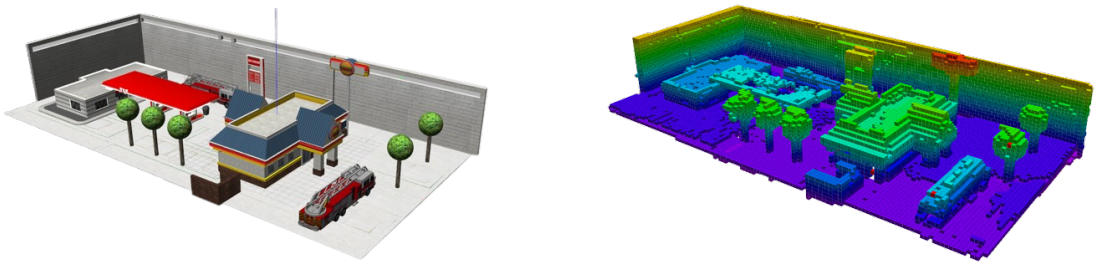


Figure 4.6: Explored map percentage (non-dashed) and distance traveled (dashed) comparison on Map 4.

mits the UAV a more extensive flight altitude range.

4.3. Outdoor Environment



(a) **(b)**

Figure 4.7: Exploration results in Outdoor Environment - a) Gazebo World, b) Resulting Octomap after Exploration using Collector Strategy

The scenario depicted in Figure 4.7 has been designed specifically to examine and leverage the 3D capabilities of the proposed approach. Accordingly, an outdoor environment is adopted where exploration locations span a larger range compared to the previous warehouse scenarios. The results of implementing the Collector strategy are depicted in Figure 4.8, presenting both the X-Y trajectory and elevation profile. It's noteworthy from this figure that there is minimal overlap between the exploration paths. Additionally, the elevation profile demon-

strates how exploration targets are set at varied altitudes, thereby requiring altitude adjustments by the UAV during the exploration process.

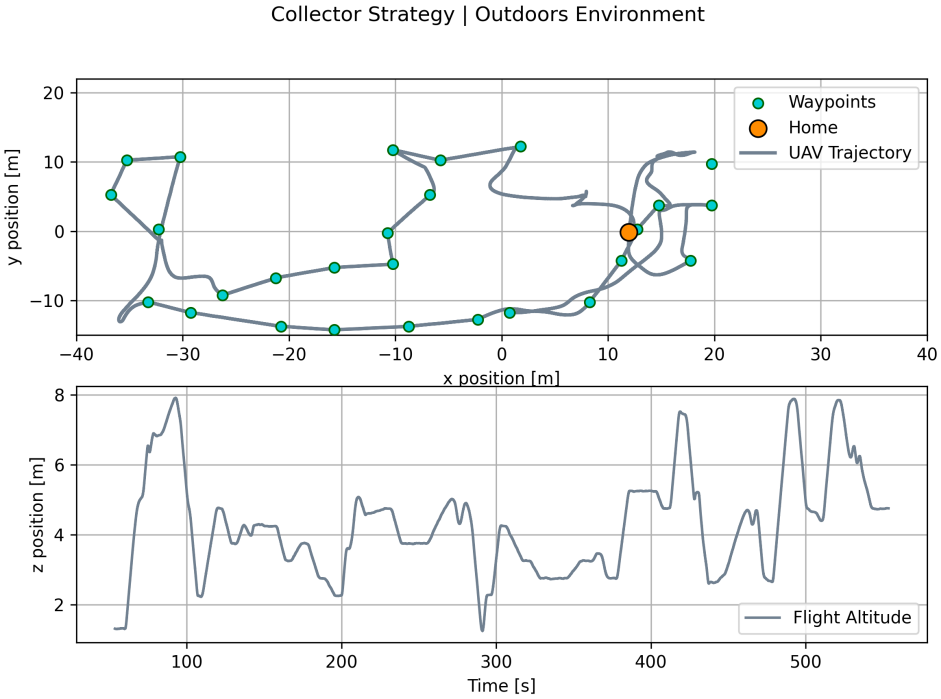


Figure 4.8: Exploration trajectories on Outdoors Environment

Lastly, Figure 4.9 presents the progression of map completion in conjunction with the corresponding distance traveled by the UAV.

4.4. Flight Testing

The flight testing phase of this project was executed in collaboration with CATEC, who generously provided their facilities, equipment, and invaluable technical expertise to facilitate the operation and testing of the system developed for this thesis project. These tests aimed to verify the proper functioning and integration of the developed exploration solution within the UAV System. Figure 4.10 illustrates the real-world environment where this project was tested. Due to safety reasons, significant limitations on the exploration area and lidar range were imposed. In this setting, figure 4.11 presents screenshots that capture the RVIZ visualization of an exploration flight inside CATEC's test bed. The left figure marks the UAV's potential exploration targets with green dots, while the pink marker represents the target selected as the best, considering its proximity to the

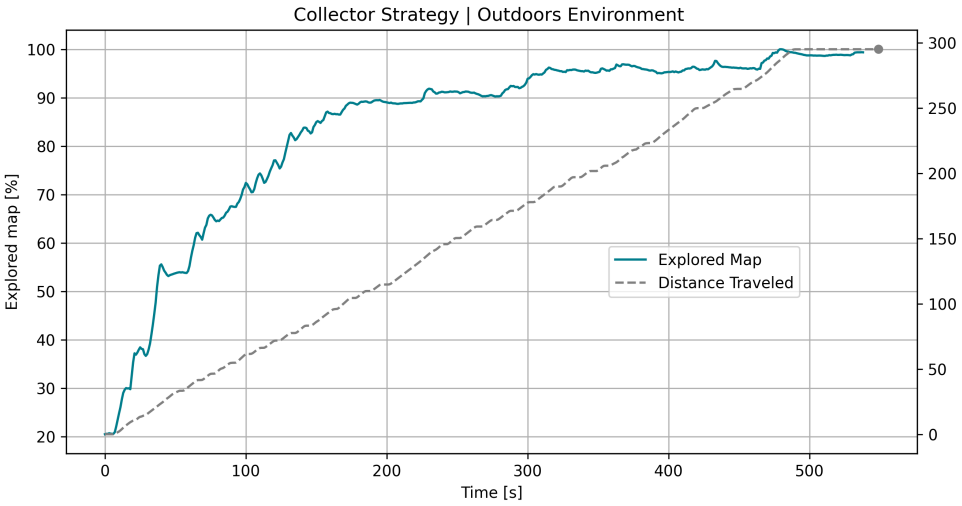


Figure 4.9: Explored map and distance traveled evaluation on Outdoors Map

UAV's current position. The right figure displays the generated octomap after visiting some exploration targets, along with the trajectory that the UAV took while exploring the test bed.



Figure 4.10: UAV quadcopter used during flight tests, Testbed from CATEC and UAV while performing autonomous exploration running onboard

Several ROS Bags were recorded from these tests to analyze the behavior and performance of the exploration strategy. Two of the most notable flights are depicted in Figure 4.12, where the X-Y plane trajectories with the visited exploration targets and the UAV's elevation profile during the flight are displayed.

Even though these were short flights, they were incredibly valuable for verifying the correct functioning of the entire exploration loop and the developed strategy, including frontier detection, selection, path planning, and navigation. It's worth emphasizing that all the algorithms ran onboard the drone, demonstrating that the computational demand does not exceed the available onboard

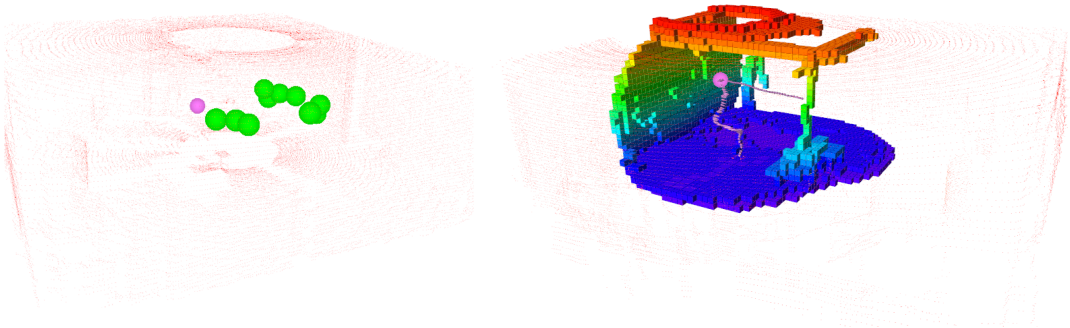
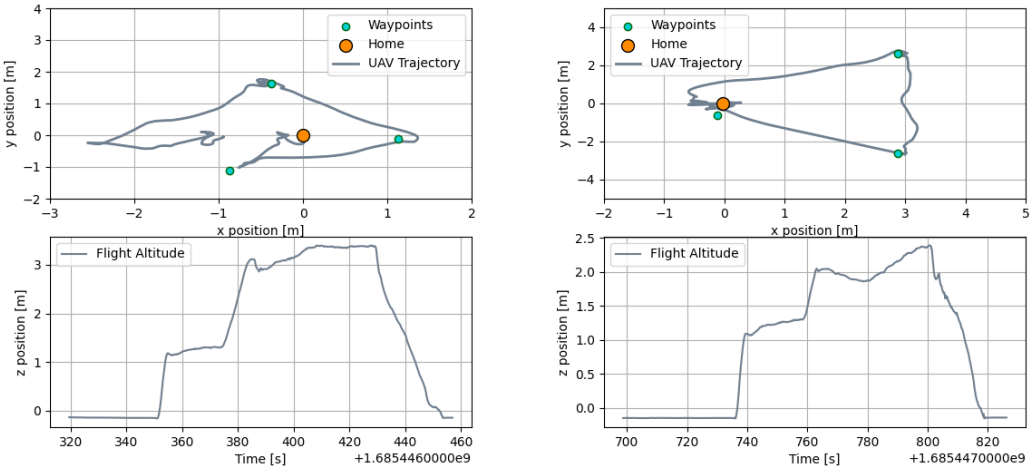


Figure 4.11: RVIZ Screenshot of the exploration flight tests in the CATEC Testbed. Left: UAV's potential and selected exploration targets. Right: Generated octomap and UAV's exploration trajectory.

computer's limitations in small UAVs.

A compilation video summarizing the work conducted during this project can be accessed via the following link. This video provides a comprehensive overview of the activities and outcomes achieved throughout the course of the project.



(a) **(b)**

Figure 4.12: Exploration trajectories extracted from ROS Bags recorded during two different flight tests

5. Conclusions

This thesis project contributes significantly to autonomous exploration with UAVs, addressing challenges inherent in the three-dimensional operating space and data complexity. It introduces an exploration algorithm designed to ensure efficient exploration while conducting an in-depth environment analysis. This strategy leverages LiDAR sensor data and integrates a frontier-based exploration technique within the OctoMap Framework. Its novelty lies in striking a balance between exploitation of known map data and exploration of unknown areas. The functionality of this algorithm has been validated both in simulated environments and real-world tests.

The project effectively develops and validates the "Collector" solution, an exploration strategy aimed at balancing exploration speed and information acquisition. This strategy handles various facets of autonomous exploration, including frontier detection, clusterization, storing, and filtering of potential exploration targets. It has demonstrated robustness and adaptability across different complexity levels of maps in the MRS-CTU simulation environment. The feasibility of implementing this algorithm onboard a small UAV has been validated through initial real-world flights in collaboration with CATEC.

The integration of a TSP solver into the proposed approach was thoroughly evaluated. This evaluation revealed a general trend where an increased number of tour points led to a longer exploration time and greater travel distance, both undesirable traits for an efficient exploration algorithm. Meanwhile, the amount of information gathered remained relatively consistent. Specifically, using Euclidean distance as the cost function for the TSP solver showed limitations, especially in environments with obstacles. Given these findings, it was concluded that the TSP solver, in its current form, did not provide a significant benefit to the exploration strategy. Consequently, the Collector approach was adapted to incorporate a policy that targets the nearest exploration point, resulting in more efficient exploration.

The Collector exploration strategy showcases its adaptability and efficiency in diverse and complex warehouse-like and outdoors environments. An exhaustive comparison with two other exploration strategies—Greedy and Multi-resolution—revealed that the Collector strategy consistently demonstrated a balance between exploration speed, completeness, and traveled distance.

As a future extension of this project, the focus should be on improving the checkers used to validate the suitability of potential exploration targets and the techniques employed to extract neighboring nodes. Despite testing the algorithm across a variety of extensive environments, the primary bottleneck lies in the methods used to identify a given node's neighbors and the additional validity checkers needed for filtering potential exploration objectives. These elements demand extra computation, potentially limiting the algorithm's applicability in larger scenarios. Currently, these processes are performed using the cartesian coordinates of the current points and their surroundings. However, conducting this search at the Key level—using the Keys (addresses) of the nodes within the Octree—could enhance efficiency.

It would also be worthwhile to implement an optimal planner, like the A* or Hybrid A*, capable of providing the shortest collision-free route. This enhancement would further improve the TSP cost matrix, allowing the closest cluster to correspond to the one requiring the shortest travel path.

Lastly, a more in-depth evaluation of the benefits of thorough exploration, as exemplified by the Collector, is yet to be conducted. For instance, cutting-edge algorithms, such as the one presented in [11], can generate an accurate 3D reconstruction of a map from a set of point cloud recordings post-exploration. This map can be colorized if an RGB camera is incorporated into the UAV.

6. Appendices

6.1. Appendix A: Detailed ROS Architecture

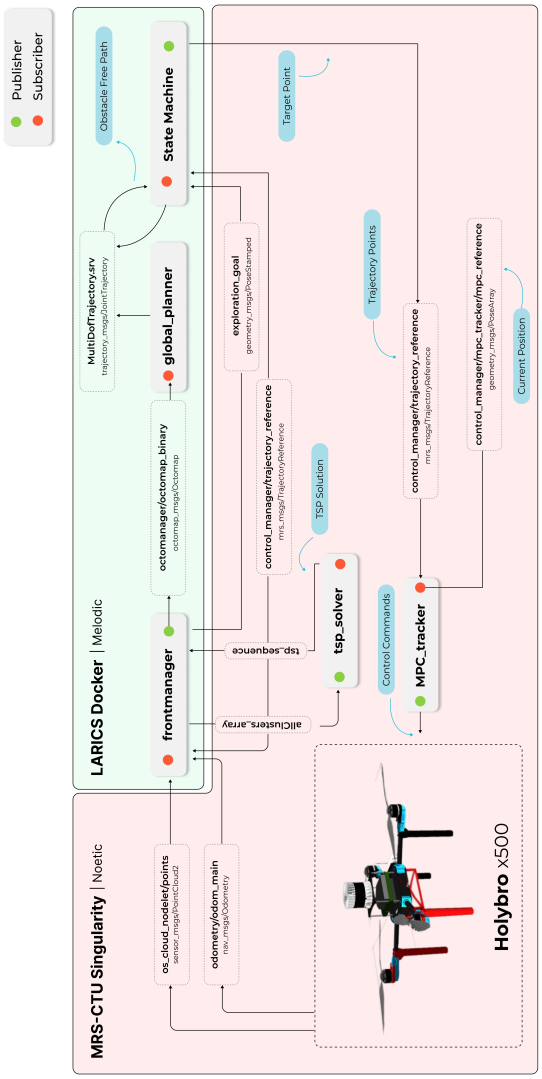


Figure 6.1: Complete ROS Implementation of the proposed UAV Autonomous Exploration.

Bibliography

[1] Ana Batinovic, Tamara Petrovic, Antun Ivanovic, Frano Petric, and Stjepan Bogdan. A multi-resolution frontier-based planner for autonomous 3d exploration. *IEEE Robotics and Automation Letters*, 6(3):4528–4535, 2021.

[2] Andreas Bircher, Mina Kamel, Kostas Alexis, Helen Oleynikova, and Roland Siegwart. Receding horizon" next-best-view" planner for 3d exploration. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1462–1468. IEEE, 2016.

[3] Titus Cieslewski, Elia Kaufmann, and Davide Scaramuzza. Rapid exploration with multi-rotors: A frontier selection method for high speed flight. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2135–2142. IEEE, 2017.

[4] CTU MRS. mrs_msgs. `https://github.com/ctu-mrs/mrs_msgs`, 2023.

[5] CTU MRS. MRS Website. `http://mrs.felk.cvut.cz/`, 2023. Accessed: May 17, 2023.

[6] Keinosuke Fukunaga and Larry Hostetler. The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on information theory*, 21(1):32–40, 1975.

[7] Keld Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European journal of operational research*, 126(1):106–130, 2000.

[8] Keld Helsgaun. LKH. `http://webhotel4.ruc.dk/~keld/research/LKH/`, 2021.

[9] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. OctoMap: An efficient probabilistic 3D mapping

framework based on octrees. *Autonomous Robots*, 2013. doi: 10.1007/s10514-012-9321-0. URL `https://octomap.github.io`. Software available at `https://octomap.github.io`.

[10] Laboratory for Robotics and Intelligent Control Systems (LARICS). larics/larics_motion_planning: A Repository for Motion Planning Algorithms. `https://github.com/larics/larics_motion_planning`, 2021. Accessed: May 17, 2023.

[11] Jiarong Lin and Fu Zhang. R 3 live: A robust, real-time, rgb-colored, lidar-inertial-visual tightly-coupled state estimation and mapping package. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 10672–10678. IEEE, 2022.

[12] Mark Moll, Ioan A. Şucan, and Lydia E. Kavraki. Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization. *IEEE Robotics & Automation Magazine*, 22(3):96–102, September 2015. doi: 10.1109/MRA.2015.2448276.

[13] Jari Saarinen, Henrik Andreasson, Todor Stoyanov, Juha Ala-Luhtala, and Achim J Lilienthal. Normal distributions transform occupancy maps: Application to large-scale online 3d mapping. In *2013 IEEE international conference on robotics and automation*, pages 2233–2238. IEEE, 2013.

[14] Lukas Schmid, Michael Pantic, Raghav Khanna, Lionel Ott, Roland Siegwart, and Juan Nieto. An efficient sampling-based method for online informative path planning in unknown environments. *IEEE Robotics and Automation Letters*, 5(2):1500–1507, 2020.

[15] Magnus Selin, Mattias Tiger, Daniel Duberg, Fredrik Heintz, and Patric Jensfelt. Efficient autonomous exploration planning of large-scale 3-d environments. *IEEE Robotics and Automation Letters*, 4(2):1699–1706, 2019.

[16] Brian Yamauchi. A frontier-based approach for autonomous exploration. In *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97.'Towards New Computational Principles for Robotics and Automation'*, pages 146–151. IEEE, 1997.

[17] Boyu Zhou, Fei Gao, Luqi Wang, Chuhao Liu, and Shaojie Shen. Robust and

efficient quadrotor trajectory generation for fast autonomous flight. *IEEE Robotics and Automation Letters*, 4(4):3529–3536, 2019.

[18] Boyu Zhou, Yichen Zhang, Xinyi Chen, and Shaojie Shen. Fuel: Fast uav exploration using incremental frontier structure and hierarchical planning. *IEEE Robotics and Automation Letters*, 6(2):779–786, 2021.

[19] Boyu Zhou, Hao Xu, and Shaojie Shen. Racer: Rapid collaborative exploration with a decentralized multi-uav system. *IEEE Transactions on Robotics*, 2023.

[20] Xiaohui Zhou, Zimu Yi, Yilin Liu, Kai Huang, and Hui Huang. Survey on path and view planning for uavs. *Virtual Reality & Intelligent Hardware*, 2 (1):56–69, 2020.

[21] Cheng Zhu, Rong Ding, Mengxiang Lin, and Yuanyuan Wu. A 3d frontier-based exploration tool for mavs. In *2015 IEEE 27th International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 348–352. IEEE, 2015.

Title

Abstract

This thesis presents the design and implementation of an Frontier-based autonomous exploration algorithm for Unmanned Aerial Vehicles (UAVs). The exploration strategy provides a careful balance between fast exploration and meticulous analysis of environments. It utilizes LiDAR sensor data within the OctoMap Framework, facilitating the frontier points detection and analysis. The developed algorithm, known as the "Collector", aims to balance exploration speed and information acquisition by combining greedy behaviors with a continuous accrual of exploration targets. The algorithm has been tested across diverse environments, showing its adaptability and efficiency in warehouse-like and outdoor settings. Although a Travelling Salesman Problem (TSP) Solver was integrated for evaluation, it was determined that its current form did not contribute substantial benefits.The findings highlight the potential of the "Collector" algorithm in real-world UAV applications, both in simulated and real-world environments.

Keywords: UAVs, Octomap, Frontier-based Exploration, LiDAR Sensor Data